A4 — Modelo lógico relacional

Convenções: a chave primária aparece sublinhada; a chave estrangeira é marcada com asterisco (*) e a relação referenciada vem indicada ao lado; (alt.)* indica chave candidata alternativa implementada por restrição `UNIQUE`.

2.1 Esquema relacional

PESSOA (id_pessoa, cpf *(alt.)*, nome, data_nascimento, sexo, email, telefone, end_logradouro, end_numero, end_complemento, end_bairro, end_cidade, end_uf, end_cep, tipo_pessoa, criado_em)

ALUNO (id_pessoa, *matricula* (alt.)*, data_ingresso, bolsista, percentual_bolsa, observacao) `id_pessoa*` → PESSOA

PROFESSOR (id_pessoa, *registro_funcional* (alt.), *titulacao*, *data_admissao*, *data_desligamento*, *id_coordenador*) `id_pessoa*` → PESSOA; `id_coordenador*` → PROFESSOR (autorrelacionamento)

RESPONSAVEL (id_responsavel, cpf *(alt.)*, nome, telefone, email, profissao)

ALUNO_RESPONSAVEL (id_aluno, *id_responsavel*, grau_parentesco, responsavel_financeiro, autoriza_saida, data_vinculo) `id_aluno*` → ALUNO; `id_responsavel*` → RESPONSAVEL

ANO_LETIVO (id_ano_letivo, ano *(alt.)*, data_inicio, data_fim, situacao)

DISCIPLINA (id_disciplina, codigo *(alt.)*, nome, serie, carga_horaria_anual, ementa, ativa) chave candidata adicional: (nome, serie)

PRE_REQUISITO (id_disciplina, *id_disciplina_requisito*, obrigatorio, observacao) ambas → DISCIPLINA (autorrelacionamento N:N)

TURMA (id_turma, id_ano_letivo, *codigo*, *serie*, *turno*, *sala*, *capacidade*, *id_professor_regente*) `id_ano_letivo*` → ANO_LETIVO; `id_professor_regente*` → PROFESSOR; chave candidata: (id_ano_letivo, codigo)

MATRICULA (id_matricula, id_aluno, *id_turma*, data_matricula, situacao, data_encerramento) `id_aluno*` → ALUNO; `id_turma*` → TURMA; chave candidata: (id_aluno, id_turma)

HISTORICO_SITUACAO_ALUNO (id_historico, id_aluno*, situacao, data_inicio, data_fim, motivo) `id_aluno*` → ALUNO; chave candidata: (id_aluno, data_inicio)

ALOCACAO (id_alocacao, id_turma, *id_disciplina*, id_professor*, carga_horaria_semanal, dia_semana, horario_inicio) `id_turma*` → TURMA; `id_disciplina*` → DISCIPLINA; `id_professor*` → PROFESSOR; chave candidata: (id_turma, id_disciplina)

AVALIACAO (id_alocacao*, numero_avaliacao, descricao, tipo, data_aplicacao, peso, valor_maximo) `id_alocacao*` → ALOCACAO (entidade fraca: identificação por dependência)

NOTA (id_matricula, *id_alocacao*, numero_avaliacao*, valor, data_lancamento, faltou, observacao) `id_matricula*` → MATRICULA; `(id_alocacao, numero_avaliacao)*` → AVALIACAO

São 14 relações: 9 derivadas de entidades e 5 derivadas de relacionamentos N:N.

2.2 Decisões de mapeamento

D1 — Especialização PESSOA → ALUNO / PROFESSOR: três tabelas

A especialização é **total e exclusiva**. Havia três estratégias possíveis:

1. **Tabela única** com todos os atributos e um discriminador. Descartada: mais da metade das colunas ficaria nula em qualquer linha (um aluno nunca tem titulação, um professor nunca tem percentual de bolsa), e as restrições `NOT NULL` específicas de cada subclasse seriam impossíveis de declarar.
2. **Uma tabela por subclasse**, sem o supertipo. Descartada por dois motivos: CPF, nome e endereço se repetiriam nas duas definições, e a unicidade global do CPF (RN01) não poderia ser garantida por uma única restrição `UNIQUE` — exigiria verificação cruzada entre duas tabelas.
3. **Três tabelas** — supertipo e duas subclasses, cada subclasse com a mesma chave do supertipo. **Adotada.**

A escolha preserva a unicidade do CPF em um só lugar, elimina colunas nulas por construção e mantém o endereço e o contato em um ponto único de manutenção. O custo é uma junção a mais sempre que se precisa do nome de um aluno ou professor — custo baixo, porque a junção é por chave primária. A **totalidade** é garantida pelo fato de que nenhuma pessoa é inserida sem que a aplicação crie simultaneamente a linha da subclasse, dentro da mesma transação, e é

verificável por consulta. A **exclusividade** é sinalizada pelo discriminador `tipo_pessoa` e verificada por consulta — o DDL declarativo do MySQL não expressa "existe em A ou em B, nunca nas duas".

D2 — Relacionamentos N:N → tabelas associativas

Todos os N:N foram mapeados para relações próprias, cada uma com seus atributos:

Relacionamento	Relação gerada	Atributos próprios
ALUNO × RESPONSAVEL	ALUNO_RESPONSAVEL	grau_parentesco, responsavel_financeiro, autoriza_saida, data_vinculo
ALUNO × TURMA	MATRICULA	data_matricula, situacao, data_encerramento
TURMA × DISCIPLINA × PROFESSOR	ALOCACAO	carga_horaria_semanal, dia_semana, horario_inicio
MATRICULA × AVALIACAO	NOTA	valor, data_lancamento, faltou, observacao
DISCIPLINA × DISCIPLINA	PRE_REQUISITO	obrigatorio, observacao

ALOCACAO merece nota: ela nasce de um relacionamento **ternário**. A restrição do domínio é que a dupla (turma, disciplina) determina o professor — uma disciplina de uma turma tem um regente único —, e por isso a chave candidata é (id_turma, id_disciplina) e não a tripla. Se a escola passasse a admitir dois professores na mesma disciplina de uma turma, bastaria estender a chave candidata para incluir o professor.

D3 — Entidade fraca AVALIACAO: identificação por dependência

AVALIACAO não recebeu chave substituta própria. Sua chave primária é `(id_alocacao, numero_avaliacao)`: o identificador parcial `numero_avaliacao` só distingue avaliações **dentro de** uma alocação. A dependência de existência é materializada por `ON DELETE CASCADE` — removida a alocação, suas avaliações deixam de fazer sentido e são removidas junto, assim como as notas que dependem delas. A alternativa (chave substituta `id_avaliacao` com `UNIQUE (id_alocacao, numero_avaliacao)`) foi descartada de propósito: ela esconderia a natureza fraca da entidade justamente no ponto em que o modelo deveria evidenciá-la.

D4 — Relacionamentos 1:1 e 1:N

Não há relacionamento 1:1 no modelo, exceto os herdados da especialização (PESSOA–ALUNO e PESSOA–PROFESSOR), que são 1:1 por definição e foram mantidos em tabelas separadas pelos motivos de D1 — fundi-los seria voltar à estratégia de tabela única.

Os 1:N foram mapeados pela via usual, com a chave estrangeira no lado N: `TURMA.id_ano_letivo`, `TURMA.id_professor_regente`, `HISTORICO_SITUACAO_ALUNO.id_aluno` e `PROFESSOR.id_coordenador`.

D5 — Chave natural × chave substituta

Relação	Chave adotada	Justificativa
PESSOA	Substituta (`id_pessoa`)	O CPF é candidato natural, mas tem onze caracteres e se propagaria por seis relações descendentes; além disso pode ser corrigido após erro de digitação, o que exigiria atualização em cascata. Mantido como chave alternativa `UNIQUE`.
ALUNO / PROFESSOR	Herdada do supertipo	Requisito da estratégia de três tabelas: a subclasse compartilha a identidade do supertipo.
RESPONSAVEL	Substituta	Mesmo argumento do CPF.
ANO_LETIVO	Substituta	O ano seria chave natural perfeitamente estável, mas a substituta uniformiza o padrão de junção; o ano permanece `UNIQUE`.
DISCIPLINA	Substituta	O código é legível, mas muda quando a escola reorganiza a matriz curricular.
TURMA	Substituta	A chave natural seria (ano_letivo, código), composta e propagável para MATRICULA e ALOCACAO.
MATRICULA	Substituta	A natural (id_aluno, id_turma) é composta e se propagaria para NOTA, que já carrega três colunas de chave. Mantida como `UNIQUE`.
ALOCACAO	Substituta	A natural (id_turma, id_disciplina) se propagaria para AVALIACAO e, por ela, para NOTA — que passaria a ter quatro colunas de chave.
ALUNO_RESPONSAVEL, PRE_REQUISITO	Natural composta	São associativas puras, sem descendentes. A chave composta expressa a regra ("um vínculo por par") sem custo algum.
AVALIACAO	Natural composta, por dependência	Ver D3.
NOTA	Natural composta	Herdada das duas relações que ela associa; expressa diretamente "uma nota por aluno por avaliação".
HISTORICO_SITUACAO_ALUNO	Substituta	A natural (id_aluno, data_inicio) funcionaria, mas a substituta simplifica a referência a um evento específico pela aplicação. Mantida como `UNIQUE`.

D6 — Atributo composto e atributo derivado

O endereço, composto no modelo conceitual, foi decomposto em colunas atômicas (`end_logradouro`, `end_numero`, ...) para atender à 1FN e permitir busca por bairro ou cidade. A idade, derivada, **não** foi armazenada: é calculada a partir de `data_nascimento` na consulta. Armazená-la criaria um dado que envelhece sozinho e entra em contradição com a data de nascimento no dia seguinte ao aniversário. Não há atributos multivalorados no modelo — os que poderiam sê-lo (telefones de contato) foram deliberadamente mantidos como único valor preferencial, decisão de escopo registrada aqui para que a ausência não pareça esquecimento.